Kaiburr: ML-KEM at (much) higher security

Anonymous Submission

Abstract. With deployment of standardized post-quantum cryptography well on the way, discussions continue around different approaches to hybrid deployments – either combining post-quantum schemes with "traditional" elliptic-curve cryptography (or RSA) or combining multiple different post-quantum schemes. The typical question asked in these discussion is what the "best" or "most conservative" approach is within a certain performance budget.

In this paper we add a data point to this discussion by investigating how to scale ML-KEM to (much) higher security levels of the underlying MLWE problem. We first show that increasing security simply by increasing the module rank k quickly reaches a limit imposed by decryption failure probability. We then propose a generalization of the noise distribution, which enables scaling to much larger module rank and MLWE hardness, while preserving the essence of ML-KEM, most notably arithmetic in the ring $\mathbb{Z}_{3329}[X]/(X^{256}+1)$.

We select three concrete parameter sets of this high-security variant of ML-KEM, which we call Kaiburr. We present benchmarking results for optimized implementations of Kaiburr on 64-bit Intel CPUs and on the Asymmetric Crypto Coprocessor (ACC) of the Pavona open-source silicon framework. Based on these benchmarks, we argue that, for example, a hybrid of FrodoKEM-976 and Kaiburr-1792 not only offers better performance than FrodoKEM-1344, it also offers protection against a broader range of hypothetical future attacks against (M)LWE.

Keywords: (M)LWE hardness, CRYSTALS-Kyber, ML-KEM, lattice-based KEMs

1 Introduction

After a multi-year, multi-round public competition, NIST published its first post-quantum standards FIPS-203 [Nat24b], FIPS-204 [Nat24a], and FIPS-205 [Nat24c] in 2024, specifying ML-KEM, ML-DSA, and SLH-DSA, respectively. Later in 2025, NIST selected an additional KEM, HQC, for standardization in the fourth round [ABC+25]. Unlike ML-KEM, which is based on Module Learning With Errors (MLWE), HQC relies on the Quasi-Cyclic Syndrome Decoding (QCSD) problem [HQC25], providing diversity beyond lattice-based assumptions. This choice also marked a shift away from the lattice-based KEMs that dominated the Round 3 submissions. Four of the schemes in the PQC Round 3 Submissions [Nat20] span a spectrum of algebraic structure, from unstructured LWE (FrodoKEM [BCD+16]), through cyclotomic modules (Kyber [BDK+18] and Saber [DKRV18]), to prime-degree number fields (NTRU LPRime [BCLvV17]). This spectrum reflects different design principles, commonly understood as structure vs. conservatism. ML-KEM's module structure gives small keys and fast NTT arithmetic, but is also seen as a potential attack surface, so conservative designs discard it entirely [BCD+16]. FrodoKEM, for example, is based on a more conservative hardness assumption of plain LWE, one of the reasons why it was recommended by bodies such as BSI, ENISA, and NLNCSA [The26] and recently standardized by ISO [ISO26].

It is useful to separate the two notions of conservatism this framing bundles together as they concern two different classes of attack. The first arises from the choice of hardness assumption. FrodoKEM rests on unstructured LWE to avoid a hypothetical structure break, an attack that would exploit the algebraic structure of ring/module LWE but not

apply directly to plain LWE. The second involves the security margin a scheme carries above its target level, which protects against advances in the best known attacks today. This could include improved lattice reduction, dual attacks, or sharper cost models that would affect the concrete security of every lattice scheme at its current parameters, including FrodoKEM. A third, orthogonal hedge is to change the hardness assumption altogether, as HQC does by relying on QCSD instead of LWE. This provides diversity, and at first glance, some additional assurance since a break of LWE could still leave HQC secure. However, it is not entirely certain that the two hardness assumptions are cryptanalytically independent. Recent work has demonstrated connections between lattice and code-based cryptanalysis, including adaptations of lattice-sieving techniques to codes [DEEK24]. The quasi-cyclic structure of HQC has also been noted to have important parallels with the ring and module structures used in structured lattice schemes [NIS25]. These approaches address different (hypothetical) failure modes: unstructured LWE protects against attacks that exploit algebraic structure, additional parameter margin protects against improvements to existing attacks, and QCSD provides diversity against a failure of the LWE assumption itself. At this time, it is impossible to predict which one of the cryptanalytic breakthroughs is more likely to occur. Additionally, hybrid deployments, which combine two KEMs so that an attacker has to break both [GHP18, BBF+19], provide an extra safety net. They offer a way to hedge against more than one of the attacks at once, so the question becomes which combination is the most prudent within a given performance budget.

Faced with many such uncertainties, increasing the margin has been a historically standard response, provided that sufficient bandwidth is available. For computationally secure schemes, a concrete security estimate is the cost of the best attack known at the time, and such estimates evolve as cryptanalysis matures. For instance, the recommended size of an RSA modulus climbed from 512 to 1024 to 2048 bits and beyond as factoring algorithms and available computation advanced [Bar20]. Lattice estimates are not static either. The cost models used for estimation and the attacks have evolved over more than a decade [CN11, APS15]. The core framework for estimating the security of LWE-based schemes has nevertheless remained stable, with BKZ-based lattice reduction at its core. Within this framework, the cost of the best known attacks continues to drift in both directions as improvements are proposed [GJ21, MAT22], scrutinized [DP23], and refined [CMST25].

If the best known attacks continue to improve, how much margin is required remains open, and the useful question is not whether today's KEMs carry enough but how much margin they can carry at all. This reframing matters because additional margin comes at a cost, requiring larger parameters and hence higher bandwidth and runtime, so no scheme can indefinitely withstand advances in cryptanalysis simply by scaling its parameters. For LWE-family KEMs there is a further constraint, since the parameters that determine security also control the decryption failure probability, so increasing the noise to gain a larger security margin necessarily affects correctness.

Crucially, what structure determines is the *price* of buying additional margin. At matched security levels, an unstructured scheme already requires roughly an order of magnitude more bandwidth than a module scheme [BCD+16, BDK+18]. Since these communication costs are considered practical enough for conservative deployments, a module scheme can spend the same bandwidth budget on larger parameters and obtain a conservative security margin while retaining the efficiency advantages of NTT arithmetic.

Recent work has explored this high-margin regime from the perspective of unstructured LWE. KyFrog [MB25] increases the LWE dimension and adjusts the noise distribution to achieve substantially higher security estimates for much larger ciphertexts. This leaves open the corresponding question for module-based KEMs:

How much additional security margin can their algebraic structure buy within the com-

munication budget of an unstructured LWE scheme, and at what computational cost?

We answer this question with KAIBURR, a module KEM that scales ML-KEM's module rank and introduces a generalized noise distribution to preserve correctness. We first characterize the correctness and security limits of naively increasing ML-KEM's module rank, then introduce a noise family that allows further scaling. We then build three different parameter sets with the new noise sampler, and implement them across conventional software and dedicated hardware-accelerated platforms. Next, we compare KAIBURR with existing KEMs, including FrodoKEM and HQC. Finally, we show that combining KAIBURR with FrodoKEM in a hybrid hedges against both structural attacks and improvements to the best known lattice attacks at lower cost than FrodoKEM's most conservative parameter set.

1.1 Our contributions

We make several contributions in this work, summarized below.

- *We show that naively scaling ML-KEM's module rank hits a correctness wall.* Raising k alone, with KYBER's noise and modulus fixed, satisfies the 2^{-128} decryption-failure bound only up to $k = 7$ and fails already at $k = 8$, so larger module rank cannot buy additional security margin on its own (Section 3).

- *We introduce a one-parameter noise family that allows us to scale further.* Our family $f(t)$ generalizes KYBER's centered binomial distribution, thinning its ± 2 tail geometrically as t grows while leaving the ± 1 mass unchanged. The parameter t can be tuned against module rank k to keep decryption failure below 2^{-128} as k increases (Section 4).

- *We select concrete parameter sets that reach large security margins within FrodoKEM-640's communication budget.* Using this trade-off, we obtain three parameter sets, KAIBURR-1792, -4608, and -6144, at $k = 7, 18, 24$, whose classical/quantum Core-SVP security climbs from 479/434 to 1717/1557 bits while their public keys and ciphertexts stay within the communication budget of FrodoKEM-640, its smallest standardized parameter set (Section 4).

- *We build KAIBURR across three implementation targets.* We implement all three parameter sets in C, in Jasmin with machine-checked constant-time verification, and as an ACC variant targeting the Pavona hardware root of trust, detailing the engineering consequences of scaling k and removing ciphertext compression (Section 5).

- *We give a formally verified bound on KAIBURR's decryption-failure probability.* Building on the EasyCrypt framework of [BKL+25], we show that removing ciphertext compression is what makes this bound provable rather than heuristic, closing a gap that exists for ML-KEM's published bound (Section 6).

- *We benchmark KAIBURR against FrodoKEM and HQC.* Across three Intel microarchitectures, we show that our AVX2 and Jasmin implementations run substantially faster than FrodoKEM-640. KAIBURR-1792 is faster than HQC-5 in every operation, and for the larger parameter sets HQC is faster in key generation and encapsulation while decapsulation is of similar cost, despite KAIBURR's (much) larger security margins (Section 7).

- *We show that a KAIBURR–FrodoKEM hybrid is a more prudent choice than FrodoKEM-1344.* A hybrid of KAIBURR-1792 and FrodoKEM-976 is smaller and about 1.7× faster than FrodoKEM-1344, retains its hedge against structural attacks, and has a much higher lattice attack cost ($2^{467.3}$ vs. $2^{281.7}$) (Section 8).

1.2 Access to artifacts

All software, including parameter analysis scripts, implementation source code, and other artifacts, is included in the artifact submitted alongside this paper. It will be publicly available under a permissive license.

2 Preliminaries

Notation. Throughout the paper, we use KYBER and ML-KEM interchangeably, except when separating the NIST submission from the standardized version. We write vectors in bold lowercase and matrices in bold uppercase, $\mathbf{x}^T$ for the transpose of $\mathbf{x}$, and $\lceil x \rfloor$ for rounding x to the nearest integer.

Ring and module setup. Let $n = 256$ and $q = 3329$, and let $R_q = \mathbb{Z}_q[X]/(X^n + 1)$ be KYBER's base ring, with coefficients centered in $(-q/2, q/2]$. The modulus satisfies $n \mid (q-1)$, needed for the number-theoretic transform (NTT) that KYBER uses for polynomial multiplication in R_q. A module rank k determines the dimensions of the public matrix $\mathbf{A} \in R_q^{k \times k}$ and of the secret and error vectors in R_q^k. $k = 2, 3, 4$ give ML-KEM-512, -768, and -1024, respectively [Nat24b].

Centered binomial noise. KYBER samples secret and error coefficients from the centered binomial distribution CBD_η. It draws 2η uniform bits $a_1, \ldots, a_\eta, b_1, \ldots, b_\eta$ and outputs $\sum_{i=1}^{\eta} a_i - \sum_{i=1}^{\eta} b_i$, a value in $\{-\eta, \ldots, \eta\}$ with variance $\eta/2$. ML-KEM uses two parameters: η_1 for the secret and error vectors $\mathbf{s}, \mathbf{e}$ in key generation and the randomness $\mathbf{r}$ in encryption, and η_2 for the encryption errors $\mathbf{e}_1, e_2$ (see below). ML-KEM-768 and -1024 use $\eta_1 = \eta_2 = 2$; ML-KEM-512 uses $\eta_1 = 3$ and $\eta_2 = 2$. CBD_η is especially desirable because it can be sampled from raw PRF output in constant time without rejection and uses only bit-counting and a subtraction.

Module-LWE. Let χ be a distribution over R_q with small coefficients. The decisional Module Learning With Errors (MLWE) problem [LS15] over R_q asks an adversary to distinguish $(\mathbf{A}, \mathbf{A}\mathbf{s} + \mathbf{e})$, where $\mathbf{A} \leftarrow R_q^{m \times k}$ is uniform and $\mathbf{s} \leftarrow \chi^k$, $\mathbf{e} \leftarrow \chi^m$, from a uniformly random pair in $R_q^{m \times k} \times R_q^m$. In ML-KEM, χ samples each coefficient from CBD_η; KAIBURR replaces it with the noise family of Section 4. Increasing k increases the dimension kn of the underlying lattice problem and is the main lever KYBER uses to raise its claimed security level [ABD+17].

The Kyber public-key encryption scheme. Key generation expands a public 32-byte seed ρ into $\mathbf{A} \in R_q^{k \times k}$, directly in the NTT domain, by rejection sampling where SHAKE-128 output is parsed into 12-bit candidates, and those smaller than q are accepted as coefficients.

- *Key generation:* sample $\mathbf{s}, \mathbf{e} \leftarrow \chi^k$ and set $\mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e}$. The public key is $(\mathbf{t}, \rho)$ and the secret key is $\mathbf{s}$.
- *Encryption* of a message $m \in \{0,1\}^n$, viewed as a polynomial with binary coefficients: sample $\mathbf{r} \leftarrow \chi^k$, $\mathbf{e}_1 \leftarrow \chi^k$, and $e_2 \leftarrow \chi$, and compute $\mathbf{u} = \mathbf{A}^T\mathbf{r} + \mathbf{e}_1$ and $v = \mathbf{t}^T\mathbf{r} + e_2 + \lceil q/2 \rfloor \cdot m$.
- *Decryption:* compute $v - \mathbf{s}^T\mathbf{u}$ and decode each coefficient to 1 if it is closer to $\lceil q/2 \rfloor$ than to 0, and to 0 otherwise.

The IND-CPA security of this scheme reduces to MLWE with χ. The public key is an instance with $m = k$, and, with $\mathbf{t}$ replaced by a uniform vector, the ciphertext is an instance with $m = k + 1$. Expanding the decryption step gives $v - \mathbf{s}^T\mathbf{u} = \lceil q/2 \rfloor \cdot m + \tilde{n}$ with noise term

$$\tilde{n} = \mathbf{e}^T\mathbf{r} - \mathbf{s}^T\mathbf{e}_1 + e_2.$$

Ciphertext compression. Instead of transmitting $\mathbf{u}$ and v at full 12-bit precision, ML-KEM compresses them by rounding each coefficient to d_u and d_v bits, respectively, with $(d_u, d_v) = (10, 4)$ for ML-KEM-512 and -768 and $(11, 5)$ for ML-KEM-1024. This reduces the ciphertext from $384(k + 1)$ to $32(d_u k + d_v)$ bytes, e.g. from 1536 to 1088 bytes for ML-KEM-768. The receiver decompresses to $\mathbf{u} + \mathbf{c}_u$ and $v + c_v$, where $\mathbf{c}_u$ and c_v are the rounding errors, so the noise term becomes

$$\tilde{n} = \mathbf{e}^T\mathbf{r} - \mathbf{s}^T\mathbf{e}_1 - \mathbf{s}^T\mathbf{c}_u + e_2 + c_v;$$

this trades a higher decryption-failure probability for a smaller ciphertext. The public key is never compressed and has $384k + 32$ bytes. Throughout the paper, "ciphertext compression" refers to this rounding of $\mathbf{u}$ and v.

From PKE to KEM. KYBER's IND-CPA-secure public-key encryption scheme is lifted to an IND-CCA2-secure KEM using a variant of the Fujisaki–Okamoto (FO) transform with implicit rejection [Nat24b]. Decapsulation decrypts, then re-encrypts the recovered message with the derived randomness, and returns a pseudorandom key if the result differs from the received ciphertext. We leave the FO transform unchanged for our construction of KAIBURR.

Decryption failure. Decryption is correct as long as every coefficient of $\tilde{n}$ is smaller than $q/4$ in absolute value. We refer to the decryption-failure probability δ as the probability, over the randomness of key generation and encryption, that at least one of the n coefficients of $\tilde{n}$ exceeds this threshold. [1] To compute δ, we adapt the KYBER team's scripts [CRY]. Each coefficient of $\mathbf{e}^T\mathbf{r}$ and of $\mathbf{s}^T\mathbf{e}_1$ is a signed sum of kn products of two independent noise coefficients. The scripts compute the distribution of a single coefficient of $\tilde{n}$ by convolution, take its tail mass beyond $q/4$, and multiply by n (a union bound over the coefficients). With compression, the rounding errors $\mathbf{c}_u$ and c_v are additionally modeled as independent of the remaining terms.

Correctness target. We require $\delta \leq 2^{-128}$ for all parameter sets.

Sia: I am not really sure how to justify this.

Core-SVP hardness estimates. We estimate MLWE hardness following the standard Core-SVP methodology [ADPS16] used throughout the KYBER submission documents and implemented in the KYBER team's public estimator scripts [CRY]. An adversary is modeled as running BKZ lattice reduction with block size b in either the primal or the dual lattice attack. The cost of BKZ-b is approximated by a single call to an SVP oracle in dimension b, whose cost is estimated as 2^{cb} operations, with $c \approx 0.292$ for classical sieving [BDGL16] and $c \approx 0.265$ for quantum sieving [Laa16]. For a given parameter set, the scripts search over block size b and number of samples m for the cheapest successful attack, and report its $\log_2$ cost as the *classical* and *quantum* Core-SVP security level. The scripts depend on the noise distribution only through its standard deviation. We report

[1] It is well established that once an attacker finds a single failure, they can use it as a compass to drastically accelerate finding more failures. This type of attack is known as "directional failure-boosting" [DRV20].

both levels throughout. For reference, the same scripts give classical/quantum Core-SVP levels of 117/106, 181/164, and 253/230 for ML-KEM-512, -768, and -1024, taking the cheaper of the primal and dual attacks as for our parameter sets.

Lattice estimator. We cross-check the Core-SVP numbers with the lattice estimator of Albrecht et al. [APS15],[2] which evaluates a wider range of attacks than the Kyber scripts like the primal uSVP and BDD attacks, the primal hybrid attack, the dual attack, the dual-hybrid attack of [MAT22], coded BKW, and the Arora–Ge attack with Gröbner bases. Its full estimate costs lattice reduction with the MATZOV model [MAT22] by default, rather than with Core-SVP. We model each parameter set as an LWE instance of dimension $256k$ with $256k$ samples, $q = 3329$, and secret and error coefficients of standard deviation $\sigma(t)$ (Section 4). For FrodoKEM, we use the estimator's built-in parameter sets, but otherwise we keep the estimator's defaults with one exception.

Raising the block-size limit. By default, the estimator has a maximum BKZ block size of 1754, at which the Core-SVP cost reaches 2^{512}, and it reports an infinite cost for any attack that would need a larger block size, so every attack on Kaiburr-4608 and Kaiburr-6144 returns an infinite cost. We raise the cap to 16384 to obtain finite estimates. The primal hybrid and BDD attacks separately hard-code the same limit of 1754 on the dimension of their final SVP call for lattices of dimension above 4096. We change this limit as well, which is the only change we make to the estimator's code. For all FrodoKEM parameters and Kaiburr-1792, the estimates are identical with and without these changes.

3 Warmup: Just changing k

The main mechanism to increase the security margin in Kyber is to raise the module rank k [ABD+17]. Fixing the ML-KEM base ring, noise distribution, and modulus, and varying only k, we measure the decryption-failure probability using the Kyber team's scripts [CRY]. Throughout we use the centered binomial noise CBD_2 (the noise of ML-KEM-768 and -1024; recall that ML-KEM-512 uses $\eta_1 = 3$). The ranks $k = 3$ and 4 recover the uncompressed analogue of standardized sets ML-KEM-768, and -1024, while $k \geq 5$ is an extrapolation. Table 1 reports the results. Against our 2^{-128} correctness target (Section 2), the bound holds at $k = 7$ ($2^{-133.7}$) but fails at $k = 8$ ($2^{-116.8}$), so raising k alone cannot buy higher security without sacrificing correctness.

These values are computed without ciphertext compression. This matches Kaiburr, which removes compression (Section 5.3), so the numbers are exact for our setting. If the compression from Kyber is kept, the rounding applied to $\mathbf{u}$ and v introduces additional error, which would increase the true failure probability, and the correctness limit on k would be reached even earlier.

4 Revisiting the noise distribution

Even though the resulting public key and ciphertext sizes at $k = 7$ remain relatively small, increasing k further is not possible without affecting correctness. Instead, one could reduce the failure probability at large k by increasing the modulus q. We note that all variants of Kyber fix $q = 3329$, which is explicitly justified in the submission documentation to NIST. The modulus $q = 3329$ is a suitable small prime satisfying the NTT-enabling condition $n \mid (q-1)$, notably weaker than the $2n \mid (q-1)$ one might expect, which Kyber deliberately does not require [ABD+19]. It was reduced from the round-1 value 7681 to

[2] https://github.com/malb/lattice-estimator, commit 53da598, with the change described below.

Table 1: Decryption-failure probability and public-key and ciphertext sizes as a function of module rank k, at fixed ML-KEM noise and modulus, without ciphertext compression. Ranks $k = 3, 4$ correspond to uncompressed ML-KEM-768, and -1024.

k	Failure probability	pk (bytes)	ct (bytes)
3	$2^{-299.6}$	1184	1536
4	$2^{-231.1}$	1568	1920
5	$2^{-186.5}$	1952	2304
6	$2^{-155.9}$	2336	2688
7	$2^{-133.7}$	2720	3072
8	$2^{-116.8}$	3104	3456

recover the bandwidth spent by dropping public-key compression and co-tuned to balance ciphertext size, failure probability, and security.

Since ML-KEM's standardization, highly optimized and high-assurance implementations have emerged across software and hardware platforms [PQ 26, BABB+23, AOP+25]. Changing q would mean not being able to reuse any existing optimized routines in software or hardware, which sacrifices an important engineering advantage and results in a different scheme altogether. We choose not to change q for this reason.

This is not to say that k is the only parameter that can be modified. We focus on changes that preserve the *essence* of KYBER: the ring $\mathbb{Z}_q[X]/(X^{256}+1)$, the NTT implementation and its constants, and the existing analysis framework. The module rank k qualifies trivially. So does the error distribution. Since the noise only affects the secret and error terms and does not interact with the arithmetic structure or the NTT, it can be replaced without altering the core implementation. What it changes are only the MLWE hardness estimate and the decryption-failure probability, both of which can be recomputed through the same scripts, over the same ring.

In this sense, the error distribution is another part of the scheme alongside k that moves both security and correctness without changing what KYBER is at its core. This is the direction we pursue and where our main contribution lies.

To scale past $k = 7$, we need a noise knob that pushes the failure-driving part of the error back down as k grows, while preserving KYBER's *essence*. Equally important, it should retain the cheap, branchless, constant-time sampling that motivates KYBER's use of a centered binomial in the first place. This requirement rules out simply using a discrete Gaussian, as in FrodoKEM.

4.1 A parameterized noise distribution

We introduce a one-parameter family $f(t)$ on $\{-2, -1, 0, 1, 2\}$, the same support as CBD_2, defined by

$$\Pr[\pm 2] = 2^{-t}, \qquad \Pr[\pm 1] = \tfrac{1}{4}, \qquad \Pr[0] = \tfrac{1}{2} - 2^{-(t-1)}.$$

The mass at ± 1 is fixed. The single parameter t acts only on the ± 2 tail, thinning it geometrically. A sample is drawn from uniform bits $a_1, \ldots, a_t$ as

$$x = (2a_t - 1)\Big(1 - a_1 + 2\textstyle\prod_{i=1}^{t-1} a_i\Big)$$

(Algorithm 1), which is branchless and therefore samples in constant time; we machine-check constant-time (CT) for all our Jasmin builds [ABB+17], and speculative constant-time (SCT) for the AVX2 builds (see Section 5.4). The standard deviation $\sigma(t) = \sqrt{0.5 + 2^{3-t}}$ decreases from 1 at $t = 4$ toward $\sqrt{0.5}$ as t grows.

Algorithm 1: Sample-$f(t)$

Input: t uniform bits
Output: $x \in \{-2, -1, 0, 1, 2\}$
Sample $a_1, \ldots, a_t \in \{0, 1\}$
$b \leftarrow \prod_{i=1}^{t-1} a_i$
$x \leftarrow (2a_t - 1)\,(1 - a_1 + 2b)$
return x

We note that $f(t)$ generalizes KYBER's own noise distribution. At $t = 4$, the probabilities become

$$\Pr[\pm 2] = \frac{1}{16}, \qquad \Pr[\pm 1] = \frac{1}{4}, \qquad \Pr[0] = \frac{3}{8},$$

which matches CBD_2, the noise distribution used by ML-KEM-768 and -1024. As $t \to \infty$, the ± 2 mass vanishes and $f(t)$ approaches CBD_1.

4.2 Finer control over the noise

A key property of $f(t)$ is that increasing t reduces the probability of the ± 2 terms exponentially while leaving the ± 1 mass unchanged. As defined above, we have

$$\Pr_{f(t)}[\pm 2] = 2^{-t}, \qquad \Pr_{f(t)}[\pm 1] = \frac{1}{4}.$$

So, t is the tunable parameter for thinning the tail of the distribution without changing the bulk ± 1 terms. This gives finer control over the noise than the standard centered binomial choices. In particular, CBD_1 and CBD_2 have standard deviations $\sqrt{0.5}$ and 1, respectively, with no centered binomial parameter providing an intermediate distribution. As shown above, $f(t)$ ranges from CBD_2 toward CBD_1 as t increases.

This finer control is useful as the module rank k increases. Increasing k increases the amount of error accumulated during encryption and therefore makes the decryption-failure constraint more restrictive. Rather than moving directly from CBD_2 to CBD_1, we can increase t as k grows, reducing the ± 2 contribution while keeping the ± 1 contribution fixed.

4.3 Selecting t

Since t is the number of uniform bits per sample, vectorization-friendly values are natural, especially for AVX2 implementations (see Section 5). As shown above, $t = 4$ recovers the CBD_2 distribution, so it serves as our anchor. Notice that $t = 8$ consumes exactly one PRF byte per coefficient, meaning it admits one-byte-per-coefficient SIMD packing without cross-byte bit-splitting. This makes $t = 8$ a very convenient choice. Our third choice, $t = 6$, sits in the middle and does not share the byte-alignment of $t = 4$ or $t = 8$. Sampling four coefficients consumes $6 \times 4 = 24$ bits, i.e. three bytes of PRF output, which is not as natural to implement as the byte-aligned $t = 4$ and $t = 8$ cases. It is nonetheless interesting relative to $t = 8$, for two reasons. First, it retains more of the ± 2 tail, giving a higher standard deviation and more security per dimension. Second, it spends fewer PRF bytes per coefficient (0.75 vs. 1), which is relevant on platforms *without* Keccak acceleration. For each choice of t, we pick the highest k satisfying the failure probability bound of 2^{-128}, staying within the communication budget of FrodoKEM-640.

We use FrodoKEM-640 as our baseline for parameter selection and comparison throughout the paper because it is the smallest standardized FrodoKEM parameter set and hence imposes the tightest communication budget of 9,616 public-key bytes and 9,720 ciphertext

bytes. This gives us a restrictive baseline for asking how much security margin KAIBURR can achieve within the communication cost of a conservative lattice-based KEM.

Table 2 lists, for every t from 2 to 8, the largest k that satisfies both constraints. The three parameter sets we carry forward are $f(4)$, $f(6)$, and $f(8)$, at $k = 7$, 18, and 24 respectively, which follow from these choices.

Table 2: Largest module rank k for each noise parameter t under the 2^{-128} failure bound and FrodoKEM-640's communication budget. Each name gives the lattice dimension $256k$. Shaded rows are the selected KAIBURR parameter sets. For $t = 8$, $k = 25$ would give a 9,632-byte public key, just over the budget. Classical/quantum security (Core-SVP) and failure probability computed with the KYBER team's scripts; other parameters as in ML-KEM, compression removed.

Scheme	k	noise	failure	class.	quant.	pk (B)	ct (B)
KAIBURR-256	1	$f(2)$	$2^{-159.1}$	52	47	416	768
KAIBURR-768	3	$f(3)$	$2^{-139.3}$	191	174	1184	1536
KAIBURR-1792	7	$f(4)$	$2^{-133.7}$	479	434	2720	3072
KAIBURR-3328	13	$f(5)$	$2^{-128.4}$	918	832	5024	5376
KAIBURR-4608	18	$f(6)$	$2^{-134.1}$	1281	1162	6944	7296
KAIBURR-5888	23	$f(7)$	$2^{-129.8}$	1650	1497	8864	9216
KAIBURR-6144	24	$f(8)$	$2^{-139.9}$	1717	1557	9248	9600

To get more conservative estimates, we also run all eight attacks of the lattice estimator (Section 2) on the three selected parameter sets. Table 3 summarizes the results. For KAIBURR-1792, the cheapest attack is the dual-hybrid attack at $2^{467.3}$, below the 479 bits from the KYBER scripts, which do not model hybrid attacks. For KAIBURR-4608 and KAIBURR-6144, the cheapest attack is BDD, at $2^{1268.9}$ and $2^{1694.3}$, which is 12 and 23 bits below their Core-SVP estimates.

As explained in Section 2, to obtain finite costs for KAIBURR-4608 and KAIBURR-6144, we had to raise the estimator's block-size limit. The cheapest attacks on these sets use block sizes of roughly 4,400 and 5,900, which are outside the range in which the MATZOV cost model was derived. Our results for KAIBURR-4608 and KAIBURR-6144 are best understood as estimates obtained from a slightly modified estimator.

Table 3: Lattice estimator results ($\log_2$ of the cost) for all eight attacks, with the block-size limit raised to 16384. The cheapest attack for each parameter set is in bold.

Attack	KAIBURR-1792	KAIBURR-4608	KAIBURR-6144
uSVP	494.0	1275.5	1701.0
BDD	488.7	**1268.9**	**1694.3**
BDD hybrid	488.7	1268.9	1694.3
BDD MITM hybrid	883.0	2411.1	3265.2
dual	519.8	1400.8	–
dual hybrid	**467.3**	1458.3	2120.4
coded BKW	–	–	–
Arora–Ge (GB)	∞	∞	∞

4.4 Sizes

Table 4 places the three KAIBURR parameter sets alongside all standardized FrodoKEM and HQC parameter sets. All three KAIBURR variants, including KAIBURR-6144, remain

within the communication budget of FrodoKEM-640, the smallest FrodoKEM parameter set. KAIBURR-1792 is smaller in combined pk+ct size than HQC-1 (5,792 vs. 6,674 bytes), and less than a third the size of HQC-5 (21,658 bytes).

Table 4: Public-key and ciphertext sizes across KAIBURR, FrodoKEM, and HQC. KAIBURR's classical/quantum Core-SVP security is as computed with the KYBER team's scripts (Table 2). The lattice estimator column gives the $\log_2$ cost of the cheapest attack found by the lattice estimator for both KAIBURR and FrodoKEM.

Scheme	Core-SVP (class./quant.)	Lattice est.[3]	pk (bytes)	ct (bytes)	pk+ct (bytes)
KAIBURR-1792	479 / 434	467.3	2720	3072	5792
KAIBURR-4608	1281 / 1162	1268.9	6944	7296	14240
KAIBURR-6144	1717 / 1557	1694.3	9248	9600	18848
FrodoKEM-640	–	162.9	9616	9720	19336
FrodoKEM-976	–	225.2	15632	15744	31376
FrodoKEM-1344	–	281.7	21520	21632	43152
HQC-1	Category 1[4]		2241	4433	6674
HQC-3	Category 3		4514	8978	13492
HQC-5	Category 5		7237	14421	21658

4.5 The ceiling of the family

The family $f(t)$ is not unbounded. The lightest noise it can produce is its limit $f(\infty)$, with $\Pr[\pm 1] = \frac{1}{4}$, $\Pr[0] = \frac{1}{2}$, $\Pr[\pm 2] = 0$, and minimal variance $\sigma^2 = 0.5$. At $n = 256$ and $q = 3329$, the failure probability crosses 2^{-128} at $k = 30$, so $k = 29$ is the largest rank that $f(t)$ can support (Table 5). Reaching higher security levels would require another change, such as changing the noise family or modifying q or n.

Table 5: Failure probability under the limiting distribution $f(\infty)$ (minimum variance $\sigma^2 = 0.5$) as k increases, uncompressed. The 2^{-128} bound is first violated at $k = 30$, so $k = 29$ is the ceiling of the $f(t)$ family at $n = 256$, $q = 3329$.

k	Failure probability	pk (bytes)	ct (bytes)
24	$2^{-158.7}$	9248	9600
26	$2^{-146.2}$	10016	10368
28	$2^{-135.5}$	10784	11136
29	$2^{-130.6}$	11168	11520
30	$2^{-126.1}$	11552	11904

[3] FrodoKEM was also evaluated using the same modified estimator with the maximum block size set to 16384 for consistency. We note that the estimated cost remains unchanged for any maximum block size setting, as long as the cost was already finite under the original estimator.

[4] The security categories for HQC are as claimed by the design team against NIST's category 1/3/5 targets. They are not directly comparable to either lattice figure, as HQC is a code-based scheme.

5 Implementation

We provide three implementations of KAIBURR, each forked from an existing ML-KEM implementation: a C implementation,[5] a Jasmin implementation with machine-checked constant-time verification,[6] and an assembly implementation for Pavona's Asymmetric Cryptography Coprocessor (ACC).[7] The ACC reuses ML-KEM's arithmetic core and accelerates Keccak in hardware. This provides us with a complementary hardware-accelerated setting where we can evaluate the cost of scaling the module rank k. As upstream, the C and Jasmin code each have a portable `ref` path and an AVX2-vectorized path; ACC has a single hand-written path.

All three implementations make the same three changes to ML-KEM:

1. the module rank k is scaled to $\{7, 18, 24\}$, giving the parameter sets KAIBURR-1792, -4608, and -6144 (named kaiburr4, kaiburr6, and kaiburr8 in the code, after their noise parameter t; see Table 2);

2. ciphertext compression is removed, so both ciphertext polynomials are serialized at full 12-bit precision; and

3. the centered binomial sampler is replaced by a sampler for the noise family $f(t)$, with $t \in \{4, 6, 8\}$.

Everything else, including the ring, the modulus $q = 3329$, the NTT and its constants, the Fujisaki–Okamoto transform, and the symmetric primitives, is left unchanged.

These modifications nonetheless require more than just mechanical changes to the code. ML-KEM implementations assume $k \leq 4$ in several places, and the AVX2 packers assume a particular coefficient order. Most of the rewritten code follows from scaling k, which we describe first (Section 5.1). We then cover the noise sampler (Section 5.2), the removal of compression (Section 5.3), and what is checked about each implementation (Section 5.4).

5.1 Scaling k

Accumulator overflow. Larger values of k can overflow the matrix-vector product. In the NTT domain, each polynomial of the product is a sum of k pointwise products, which ML-KEM accumulates in an `int16_t` and reduces once at the end. In the C reference, the C AVX2, and the Jasmin reference code, each output coefficient is the sum of two separately Montgomery-reduced products, so it is bounded by $2(q-1) = 6656$ rather than by q, and the sum of k terms fits only while $2kq < 2^{15}$, i.e. $k \leq 4$. On the other hand, the Jasmin AVX2 code accumulates both products at 32 bits and applies a single Montgomery reduction, so its terms are bounded by q and it is safe to $k \leq 9$; it also fails at $k = 18$ and $k = 24$.

The C and Jasmin implementations therefore Barrett-reduce the accumulator after every pointwise product, at a cost of one extra reduction per term.[8] ACC needs no such change since it accumulates with `bn.addvm`, a modular addition against the `MOD` register, which is set to $2q$ throughout the basemul. Its accumulator stays below $2q$ for every k, and the reduction is free.

[5] Forked from the KYBER reference and AVX2 implementations, https://github.com/pq-crystals/kyber, commit `4768bd3`.

[6] Forked from `formosa-mlkem`, https://github.com/formosa-crypto/formosa-mlkem, commit `475b874`.

[7] https://github.com/pavona/pavona, commit `07ccb24`.

[8] We note that reducing after every term is more conservative than necessary, since a Barrett reduction leaves headroom for several further terms. We did not implement this, as reduction is not a significant part of the cost.

Matrix generation. The AVX2 matrix sampler in ML-KEM hardcodes how the k^2 matrix entries are grouped into batches of four for the 4-way Keccak for each $k \leq 4$. We replace this with a generic loop; it samples four entries at a time with 4-way Keccak and handles any remaining entries with a scalar tail. Only $k = 7$ has a remainder, of one entry.

The same loop generates both $\mathbf{A}$ for key generation and its transpose $\mathbf{A}^T$ for encapsulation, by selecting the matching domain separators. The C code computes these separators from the row and column indices, while the Jasmin code reads them from a precomputed table. ACC generates one entry at a time and derives each separator from a running nonce, with separate handling for the transposed and non-transposed orders.

Memory. The C and Jasmin code store the full matrix $\mathbf{A}$ on the stack. Its size grows quadratically with k, from 24.5 KiB at $k = 7$ to 288 KiB at $k = 24$. This is not suitable for ACC, whose stock configuration only provides 32 KiB of data memory. That is less than the stack frame of a single decapsulation at $k = 24$, which is about 34 KiB even without storing the matrix, before the input ciphertext and secret key are counted.

We address this in two ways. First, the ACC implementation never stores $\mathbf{A}$. It generates one polynomial at a time, multiplies it into a running accumulator, and discards it. Second, we enlarge ACC's visible data memory from 31 KiB to 127 KiB in the hardware description, out of 128 KiB of physical DMEM including a 1 KiB scratch region. The ACC results in Section 7 are therefore for this modified hardware configuration.

5.2 The noise sampler

The noise family $f(t)$ maps t uniformly random bits $a_1, \ldots, a_t$, read least-significant first, to the coefficient

$$(2a_t - 1)\left(1 - a_1 + 2\prod_{i<t} a_i\right).$$

All our implementations replace the centered binomial sampler with this expression, and the PRF output grows to $t \cdot n/8$ bytes per polynomial. The five variants differ in how they compute two parts of the expression: the product term $\prod_{i<t} a_i$ and the sign $(2a_t - 1)$. Table 6 summarizes their choices.

Table 6: How each implementation computes the product term and applies the sign in $f(t)$.

Implementation	Product term	Sign	Word size
C ref	bit-by-bit AND chain	integer multiply	scalar
C AVX2	compare against a mask	conditional lane negate	8–32-bit lanes
Jasmin ref	carry propagation	conditional two's-complement	scalar
Jasmin AVX2	compare against a mask	conditional two's-complement	8–16-bit lanes
ACC	carry propagation	modular subtract + blend	16-bit lanes

The Jasmin reference code and the ACC assembly avoid the literal AND chain with a carry trick. Adding one to the low $t-1$ bits carries into bit $t-1$ exactly when all of those bits are set, so a right shift gives us the product. The AVX2 kernels instead isolate the same bits and apply a vector compare, which gives the product directly as an all-ones or all-zero mask. In every vectorized and assembly variant, we then apply the sign separately and without branching.

Extracting t-bit fields from the PRF output is simplest for $t = 8$ and $t = 4$, which align with bytes and nibbles, and hardest for $t = 6$, as explained in Section 4. Both AVX2 kernels use an extra shuffle or gather for this step. The C kernel also reads slightly beyond the end of the PRF output, so the buffer includes 32 bytes of trailing slack. ACC lacks a

byte-shuffle instruction, so it instead uses a sequence of `bn.rshi` shifts to move the 6-bit fields across register boundaries.

Key generation needs $2k$ noise polynomials. The AVX2 code samples them in groups of four using the 4-way Keccak.

5.3 Removing compression

KAIBURR serializes both ciphertext polynomials $\mathbf{u}$ and v at full 12-bit precision, so the ciphertext grows to $(k+1) \cdot 384$ bytes. The public key keeps its size of $384k + 32$ bytes, since ML-KEM never compresses it.

The reference code and ACC reuse their existing 12-bit packer unchanged. The AVX2 code needs a new one. Its existing packer is used only for the public key, which is stored in the permuted coefficient order of the AVX2 NTT, so it applies that permutation while packing. Ciphertext polynomials leave the inverse NTT in standard order. Both AVX2 forks therefore add a packer and unpacker that work in standard order, without the permutation.

The new packer emits 24 bytes per iteration as two 16-byte stores twelve bytes apart, so its final iteration would write past the end of the buffer. The C code therefore writes every iteration to a scratch buffer and copies 24 bytes out, and the Jasmin code handles the final iteration as a separate case.

Removing compression also has a consequence beyond engineering. It makes the decryption-failure probability formally provable, which we develop in Section 6.

5.4 Assurance

Jasmin. The Jasmin implementation is a fork of `formosa-mlkem` [BABB+23], whose EasyCrypt proofs establish functional correctness for ML-KEM. These proofs do not carry over to KAIBURR, since the sampler, module rank, and serialization all change. What we check is much narrower.

Every exported function carries a security-type annotation. We check all six builds with Jasmin's constant-time checker for constant-time (CT). The three AVX2 builds, whose exported functions carry `sct` annotations, are additionally checked for speculative constant-time (SCT) [SBG+23], which also rules out leaking secrets through instructions executed speculatively after a mispredicted branch (Spectre v1). As upstream, the reference builds carry `ct` annotations and are checked for CT only.

Testing. The C implementation relies on the deterministic round-trip self-tests inherited from its upstream ML-KEM codebase. The Jasmin implementation runs the same kind of self-test, and additionally cross-checks its output against the C implementation, bit-for-bit, across all six builds. The ACC implementation is validated against a Python model based on `kyber-py`[9] which subclasses `ML_KEM`, overrides the noise sampler, and disables ciphertext compression. For each parameter set, the model generates known-answer tests (KATs) for key generation, encapsulation, and decapsulation (including implicit rejection), as well as round-trip tests. Unlike the Jasmin code, ACC's constant-time behavior is not machine-checked; the hand-written kernels avoid secret-dependent branches by construction.

6 Correctness

In the previous sections, we discussed removing compression as a design choice with engineering consequences. We now show why that change matters beyond engineering and

[9] https://github.com/GiacomoPope/kyber-py

show that it turns the decryption-failure probability from a heuristic estimate into a formally verified bound.

Why compression is the obstruction. In the noise expression that Barbosa et al. [BKL+25] derive for ML-KEM, $\tilde{n} = \mathbf{e}^T\mathbf{r} - \mathbf{s}^T\mathbf{e}_1 - \mathbf{s}^T\mathbf{c}_u + e_2 + c_v$ (Section 2), the rounding errors $\mathbf{c}_u$ and c_v are the only source of difficulty. First, because the correctness adversary holds the secret key, no direct MLWE reduction is available which forces a worst-case elimination of c_v and a split into two sub-bounds. Second, $\mathbf{c}_u$'s own rounding-error distribution is not symmetric and so fails the "good" property their framework requires. This forces an index-dependent, 256-term sum rather than a single closed-form union bound. In the case of FrodoKEM, since it admits no ciphertext compression, the noise term only involves good distributions, which gives a clean union bound with no heuristic assumption. The cost of compression is concrete and quantitative. For ML-KEM-768, the fully provable bound (which pays for compression) is 2^{-80}, while the heuristic bound that just assumes rounding-of-uniform is 2^{-164}, a gap of roughly 84 bits.

Consequence for Kaiburr We adapt the EasyCrypt framework of [BKL+25] and instantiate it for Kaiburr which has no ciphertext compression like FrodoKEM. We reuse their theory for combining distributions over a ring and their proofs that the relevant properties are preserved. We also use their reduction to a single coefficient and their OCaml implementation for computing a sound upper bound on the resulting tail probability. Kaiburr retains Kyber as specified in Sections 2–5 in every respect the analysis depends on. The only property of the noise family $f(t)$ the proof uses is that it is "good" in their sense: non-trivial and symmetric around 0 (see Section 4).

Since the rounding terms vanish, the noise collapses to $\tilde{n} = \mathbf{e}^T\mathbf{r} - \mathbf{s}^T\mathbf{e}_1 + e_2$. As a result, every coefficient is identically distributed independent of its index, and the union bound becomes a single factor of 256 rather than a sum of 256 distinct terms. Moreover, no additional MLWE term is needed.

Computing the bound. The tail probability is computed outside EasyCrypt by OCaml code that mirrors the EasyCrypt expression. We report the results for Kaiburr-1792 ($k = 7$), Kaiburr-4608 ($k = 18$), and Kaiburr-6144 ($k = 24$) in Table 7, alongside the failure probability obtained for each parameter set from the Kyber team's estimator scripts (Table 2).

Table 7: Decryption-failure probability for each Kaiburr parameter set: the formally verified bound obtained in this work, and the estimate from the Kyber team's scripts (Table 2).

Scheme	k	Verified bound (this work)	Kyber-script estimate
Kaiburr-1792	7	$2^{-133.3}$	$2^{-133.7}$
Kaiburr-4608	18	$2^{-133.8}$	$2^{-134.1}$
Kaiburr-6144	24	$2^{-139.6}$	$2^{-139.9}$

To put the numbers in context, for ML-KEM-768, Barbosa et al. [BKL+25] compute a heuristic bound of 2^{-164}, while the best bound they can justify against the definition of correctness is only 2^{-80} with a gap of roughly 84 bits. For Kaiburr there is no such gap to close, since removing compression leaves no rounding term for a heuristic assumption to apply to. Table 7 shows our provable bound closely tracks the Kyber team's own estimate for every parameter set. This shows that the bound is provable as well as tight in practice.

The price KAIBURR pays by removing compression is ciphertext size. However, this additional size is modest in the context of the high-security parameter sets considered in this work; it also remains within our fixed communication budget.

7 Benchmarks

We benchmark the C and Jasmin implementations on three Intel microarchitectures: Skylake, Coffee Lake, and Raptor Lake, with Hyper-Threading and Turbo Boost disabled. For the hybrid Raptor Lake processor, we pin the benchmark to a performance core. All code is compiled using `GCC 12.2.0`.

We compare the KAIBURR variants with FrodoKEM and HQC. We use the SHAKE-based variant of FrodoKEM as the primary comparison baseline. Both FrodoKEM-SHAKE and KAIBURR use SHAKE-based symmetric primitives. However, FrodoKEM-AES can take advantage of dedicated AES instructions on our Intel processors, which introduces an implementation-specific hardware advantage unrelated to the lattice-based KEM operations we are looking to compare.[10] We mainly compare against FrodoKEM-640, the smallest standardized FrodoKEM parameter set, since it is already substantially more expensive than KAIBURR. On the i7-8700K, we additionally report FrodoKEM-976 and FrodoKEM-1344, which we use in the hybrid comparison of Section 8.

All three KEMs are evaluated using a common benchmark driver adapted from the implementation used in [BABB+23]. We first perform 1000 untimed warm-up iterations, followed by 10001 timed iterations. We report the median execution time and interquartile range (Q_1–Q_3). This procedure is repeated three times, and we report the results from the run whose median is itself the median of the three runs, together with its corresponding quartiles. Each iteration verifies that the decapsulated shared secret matches the encapsulated shared secret.

Randomness comes from Linux `getrandom`. FrodoKEM and KAIBURR call it within `keypair` and `enc`, so that syscall is inside the measured region. HQC instead seeds a SHAKE-256 PRNG once, which we seed from `getrandom` before timing begins.

The benchmark results are reported separately for each processor. Results for the Intel Core i7-6700K, Intel Core i7-8700K, and Intel Core i9-13900K are given in Tables 8, 9, and 10, respectively.

KAIBURR is faster than FrodoKEM-640 across every **operation**, **parameter set**, **implementation**, and **machine**. KAIBURR-1792 is 21.5–28.0x faster, KAIBURR-4608 is 4.2–5.3x faster, and KAIBURR-6144 is 2.4–3.1x faster than FrodoKEM-640.

Comparing the KAIBURR C/asm and Jasmin implementations, the faster implementation depends on the microarchitecture. On the i7-6700K and i7-8700K, Jasmin is faster than C/asm for every parameter set and every operation, by 3.6–11.6% and 5.4–13.1%, respectively. On the i9-13900K, this reverses and C/asm is faster than Jasmin for every parameter set and every operation, by 9.2–14.0%.

Across parameter sets, C/asm KeyGen cycle counts scale by a factor of 5.19 (6700K) / 5.22 (8700K) / 5.32 (13900K) from KAIBURR-1792 ($k = 7$) to KAIBURR-4608 ($k = 18$), and by a factor of 1.68 (6700K) / 1.68 (8700K) / 1.69 (13900K) from KAIBURR-4608 to KAIBURR-6144 ($k = 24$). For reference, k itself grows by factors of 2.57 and 1.33 over these two steps, and k^2 by factors of 6.61 and 1.78.

KAIBURR-1792 is faster than HQC-5 in every operation on all three machines; for the C/asm implementation, by 64.0–90.0% on the 6700K (KeyGen, Enc, Dec), 64.5–89.8% on the 8700K, and 67.8–90.1% on the 13900K. Comparing each KAIBURR parameter set to the HQC parameter set of the same index (KAIBURR-1792/HQC-1, KAIBURR-4608/HQC-3, KAIBURR-6144/HQC-5), HQC's KeyGen is faster than the corresponding KAIBURR

[10] Additionally, we do not report a direct cycle-count comparison with KyFrog [MB25], the KEM closest to KAIBURR in principle, since no public hand-optimized implementation of the scheme exists.

Table 8: Benchmark results in cycles on an Intel Core i7-6700K. We report only the AVX2 implementations. The reference implementations are, on average, 545.7 percent slower for Kaiburr C and 637.8 percent slower for Kaiburr Jasmin, measured using the geometric mean. Values are reported as median (Q1–Q3).

KEM	Impl.	KeyGen	Enc	Dec
Kaiburr-1792	C/asm	156186 (155490–157822)	160062 (159576–161914)	171888 (171050–173356)
Kaiburr-1792	Jasmin	146528 (145948–147226)	148044 (147606–148788)	151990 (151322–152728)
Kaiburr-4608	C/asm	809842 (807208–813058)	817844 (815224–821062)	845458 (842838–848794)
Kaiburr-4608	Jasmin	780574 (779084–782266)	781302 (779932–782906)	796676 (795256–798396)
Kaiburr-6144	C/asm	1361664 (1358070–1366138)	1365370 (1361692–1369542)	1407632 (1403884–1411932)
Kaiburr-6144	Jasmin	1308772 (1306774–1311182)	1309238 (1307196–1312060)	1324338 (1322338–1326730)
FrodoKEM-640	[Fro]	3860180 (3859268–3868164)	4077768 (4076966–4087064)	4044176 (4043428–4053764)
HQC-1	[HQC]	88162 (88006–89002)	172786 (172618–173672)	405674 (405402–406028)
HQC-3	[HQC]	209986 (209758–210708)	407810 (407430–408654)	858366 (857904–859064)
HQC-5	[HQC]	433948 (433570–434640)	847696 (846974–848576)	1712396 (1711782–1713680)

KeyGen by a factor of 1.77, 3.86, and 3.14 on the 6700K, 1.73, 3.79, and 3.11 on the 8700K, and 1.68, 3.61, and 2.90 on the 13900K.

Interquartile ranges are narrow relative to the medians across all measurements, typically within 1–2% of the median (e.g., for Kaiburr-1792 C/asm, 1.3–1.5% on the 6700K and 1.3–1.4% on the 8700K).

We additionally benchmark the ACC implementation described in Section 5 on Pavona's cycle-accurate RTL simulator for the accelerator. Because the simulator is deterministic for a fixed random seed, repeated runs of the same test are identical in cycles. Therefore unlike the CPU benchmarks above, we report a single exact cycle count per operation rather than a median and interquartile range. We measure each KEM operation within a single keypair–encapsulate–decapsulate roundtrip and record cumulative execution statistics after each operation. The individual costs are then recovered by subtraction: KeyGen $= M_1$, Enc $= M_2 - M_1$, and Dec $= M_3 - M_2$. Results are shown in Table 11.

Table 12 shows the three functions accounting for the largest share of cycles in the same roundtrip benchmark, obtained from the simulator's per-function execution statistics. Across all three parameter sets, matrix generation (`poly_gen_matrix`) dominates, consuming 73–84% of total cycles.

`poly_gen_matrix` expands the public seed into the uniformly random matrix $\mathbf{A}$ via rejection sampling on SHAKE-128 output. Since it dominates the ACC cycle counts, we break down its costs further using the simulator's per-instruction execution statistics. For Kaiburr-6144, this instruction-level breakdown of `poly_gen_matrix` shows that rejection sampling is responsible for the vast majority of its cost. Only 464,022 of its 6,235,212 cycles (7.4%, or 6.2% of the entire benchmark) are spent stalled on `bn.wsrr` waiting for the hardware SHAKE-128 core to produce the next squeeze block. The remaining 92.6%

Table 9: Benchmark results in cycles on an Intel Core i7-8700K. We report only the AVX2 implementations. The reference implementations are, on average, 540.8 percent slower for KAIBURR C and 653.8 percent slower for KAIBURR Jasmin, measured using the geometric mean. Values are reported as median (Q1–Q3).

KEM	Impl.	KeyGen	Enc	Dec
KAIBURR-1792	C/asm	156156 (155579–157778)	159719 (159235–161416)	173015 (172106–174398)
KAIBURR-1792	Jasmin	143857 (143402–144559)	144987 (144656–145831)	150435 (149704–151098)
KAIBURR-4608	C/asm	814685 (812084–817870)	821346 (818719–824533)	848608 (845755–852000)
KAIBURR-4608	Jasmin	762767 (761322–764429)	765475 (764033–767156)	781079 (779648–782776)
KAIBURR-6144	C/asm	1367405 (1363808–1371615)	1376530 (1372953–1380951)	1411130 (1407567–1415488)
KAIBURR-6144	Jasmin	1293036 (1291067–1295424)	1295674 (1293713–1298070)	1313175 (1311174–1315640)
FrodoKEM-640	[Fro]	3860446 (3857908–3864447)	4062824 (4061683–4069217)	4029412 (4027484–4035297)
FrodoKEM-976	[Fro]	8270912 (8266234–8274033)	8528877 (8523192–8530688)	8411864 (8402988–8419383)
FrodoKEM-1344	[Fro]	14508496 (14505193–14510927)	14790823 (14788919–14793792)	14614161 (14611158–14617148)
HQC-1	[HQC]	90229 (90070–90974)	172063 (171888–172892)	404274 (404018–404596)
HQC-3	[HQC]	215176 (214956–215857)	404687 (404324–405517)	857576 (857048–858323)
HQC-5	[HQC]	439786 (439458–440546)	838353 (837433–839669)	1698485 (1697851–1699766)

is software cost of the rejection-sampling loop itself. 83.6% of `poly_gen_matrix`'s cycles (69.9% of the whole benchmark) are retired arithmetic instructions (`bn.and`, `bn.cmp`, and `bn.sel`) performing the mask/compare/accumulate operations of rejection sampling, and a further 8.5% (7.1% of the whole benchmark) comes from branch overhead (`bne` and `beq`) in the accept/reject loop. We repeated this instruction-level breakdown for KAIBURR-1792 and KAIBURR-4608 and found the same pattern. The hardware-Keccak-wait share stays flat (5.5% and 6.0% of each benchmark's total cycles, respectively, versus 6.2% for KAIBURR-6144), while the rejection-sampling arithmetic share grows with the parameter set (60.6% and 67.6%, versus 69.9% for KAIBURR-6144). Table 13 gives the full comparison.

8 A hybrid approach

As discussed in Section 1, a larger security margin and the removal of algebraic structure address different hypothetical classes of attack, representing different design principles. At this time, it is unclear which of the two stances on conservatism should be prioritized. A hybrid KEM does not require choosing between them. Hybrid KEMs run two KEMs side by side and derive a single shared key from both shared secrets and both ciphertexts; with an appropriate combiner, the result is IND-CCA secure as long as at least one component is [GHP18, BBF+19].

Table 10: Benchmark results in cycles on an Intel Core i9-13900K. We report only the AVX2 implementations. The reference implementations are, on average, 498.2 percent slower for Kaiburr C and 557.8 percent slower for Kaiburr Jasmin, measured using the geometric mean. Values are reported as median (Q1–Q3).

KEM	Impl.	KeyGen	Enc	Dec
Kaiburr-1792	C/asm	121724 (120872–122872)	125204 (124310–126302)	134816 (134058–136082)
Kaiburr-1792	Jasmin	141364 (140882–142142)	143058 (142620–143842)	148502 (147944–149212)
Kaiburr-4608	C/asm	648064 (645678–650674)	654366 (651728–657234)	676840 (674446–679554)
Kaiburr-4608	Jasmin	747376 (745792–749120)	749472 (747868–751250)	764490 (762874–766280)
Kaiburr-6144	C/asm	1095102 (1091646–1098922)	1097334 (1093566–1101348)	1127588 (1124030–1131712)
Kaiburr-6144	Jasmin	1272644 (1270318–1275208)	1272480 (1270246–1274906)	1285796 (1283420–1288436)
FrodoKEM-640	[Fro]	3112706 (3107260–3124744)	3239826 (3233936–3249204)	3195408 (3187982–3208002)
HQC-1	[HQC]	72578 (71902–73220)	139896 (138680–141026)	312692 (310798–314622)
HQC-3	[HQC]	179494 (178058–180668)	332806 (329936–335788)	668334 (663984–672664)
HQC-5	[HQC]	377670 (376224–380158)	705038 (700890–709262)	1357572 (1350500–1364182)

Table 11: Benchmark results in cycles for the ACC implementation on Pavona's cycle-accurate RTL simulator. Values are exact cycle counts.

KEM	KeyGen	Enc	Dec
Kaiburr-1792	240816	239348	257474
Kaiburr-4608	1453490	1424342	1467109
Kaiburr-6144	2505793	2449864	2506070

We consider a hybrid of Kaiburr-1792 and FrodoKEM-976 and compare it with FrodoKEM-1344, the most conservative standardized FrodoKEM parameter set. We have not implemented this hybrid as a single KEM. Its sizes in Table 14 are exact, since they are the sums of the component sizes. Its cycle counts in Table 15 are the sums of the measured component medians and do not include the combiner, which hashes both shared secrets and both ciphertexts.

The hybrid is smaller than FrodoKEM-1344, by 14.7% in public-key size and 13.0% in ciphertext size, and the sum of its component cycle counts is about 1.7× lower. Almost all of this speedup comes from FrodoKEM-976 itself, and Kaiburr-1792 accounts for about 2% of the hybrid's cycles.

A hybrid KEM is as hard to break as the stronger component. Against improvements to lattice attacks in general, this is Kaiburr-1792: its cheapest known lattice attack costs $2^{467.3}$ compared with $2^{281.7}$ for FrodoKEM-1344. Against an attack that exploits the module structure of Kaiburr but does not apply to plain LWE, the hybrid retains a structural hedge, but at the level of FrodoKEM-976 ($2^{225.2}$) rather than FrodoKEM-1344 ($2^{281.7}$). The hybrid is therefore preferable to FrodoKEM-1344 when this level of

Table 12: The three functions with the largest share of total cycles in the ACC roundtrip benchmark, ranked 1st to 3rd, obtained from the simulator's per-function execution statistics. All values are percentages of each variant's total roundtrip cycles. Cell shading is scaled to the largest value in the table.

KEM	1st	2nd	3rd
KAIBURR-1792	`poly_gen_matrix` 72.60	`basemul_acc` 8.08	`f4` 6.26
KAIBURR-4608	`poly_gen_matrix` 80.78	`basemul_acc` 9.23	`f6` 3.96
KAIBURR-6144	`poly_gen_matrix` 83.56	`basemul_acc` 9.57	`f8` 2.05

Table 13: Cycle breakdown of `poly_gen_matrix` across parameter sets, obtained from the simulator's per-instruction execution statistics. All values are percentages of each variant's total roundtrip cycles. Cell shading is scaled to the largest value in the table.

KEM	Rejection sampling	Branch overhead	Keccak wait	Memory latency
KAIBURR-1792	60.6	6.2	5.5	0.3
KAIBURR-4608	67.6	6.8	6.0	0.4
KAIBURR-6144	69.9	7.1	6.2	0.4

protection against structural attacks is considered sufficient. A deployment that wants no reduction in structural margin can instead add KAIBURR-1792 to FrodoKEM-1344; by the same component measurements, this costs about 1% more cycles before the combiner and 13–14% more bandwidth.

9 Discussion

Our results show that KAIBURR remains computationally practical at much higher security margins. In our Intel benchmarks, every KAIBURR parameter set is substantially faster than FrodoKEM-640 in every operation. This includes our largest parameter set, KAIBURR-6144, which stays within FrodoKEM-640's communication budget, reaches CoreSVP estimates of 1717 bits classical and 1557 bits quantum, and is still 2.4–3.1 times faster.

A gap this wide reflects the computational advantage of retaining module structure. It also shows that the bandwidth separating the two schemes can be spent in more than one way. Even ML-KEM-1024, the largest ML-KEM parameter set, has a public key 8,048 bytes and a ciphertext 8,152 bytes smaller than FrodoKEM-640, the smallest FrodoKEM parameter set. FrodoKEM spends that bandwidth on *removing* algebraic structure, whereas KAIBURR spends it on a larger security margin while *retaining* the structure. Framed this way, our work also raises a practical question for deployment. FrodoKEM-640's communication and computational costs are already considered acceptable for conservative deployments, such as those following the recommendations discussed in Section 1. Could a deployment that accepts these costs instead spend them on a high-margin variant of ML-KEM? Our work suggests that it can. Naively increasing the module rank reaches a correctness limit (Section 3), but with KAIBURR's noise distribution, the same budget supports module ranks up to 24 and much higher security estimates, while every parameter set remains faster than FrodoKEM-640. Notably, the budget can also be split between the two by using a hybrid of KAIBURR-1792 and FrodoKEM-976, which has a smaller public key and ciphertext than FrodoKEM-1344 (Section 8).

We find it important to stress that these two approaches remain complementary

Table 14: FrodoKEM-1344 versus a hybrid of Kaiburr-1792 and FrodoKEM-976. The hybrid's sizes are the sums of its two components. Lattice attack costs are $\log_2$ of the cheapest attack found by the lattice estimator.

	FrodoKEM-1344	Kaiburr-1792 + FrodoKEM-976
pk (bytes)	21 520	18 352
ct (bytes)	21 632	18 816
Cheapest lattice attack	$2^{281.7}$	$2^{467.3}$
if module structure is broken	$2^{281.7}$	$2^{225.2}$

Table 15: Cycle counts on an Intel Core i7-8700K, AVX2 implementations, measured as in Section 7. Values are median (Q1–Q3). The rows are the sums of the Kaiburr-1792 and FrodoKEM-976 medians and exclude the combiner.

KEM	Impl.	KeyGen	Enc	Dec
Kaiburr-1792	C/asm	156156 (155579–157778)	159719 (159235–161416)	173015 (172106–174398)
Kaiburr-1792	Jasmin	143857 (143402–144559)	144987 (144656–145831)	150435 (149704–151098)
FrodoKEM-976	[Fro]	8270912 (8266234–8274033)	8528877 (8523192–8530688)	8411864 (8402988–8419383)
Kaiburr-1792 + FrodoKEM-976	C/asm	8427068	8688596	8584879
Kaiburr-1792 + FrodoKEM-976	Jasmin	8414769	8673864	8562299
FrodoKEM-1344	[Fro]	14508496 (14505193–14510927)	14790823 (14788919–14793792)	14614161 (14611158–14617148)

rather than substitutes, since they address different attacks. As discussed in Section 1, Kaiburr's larger margin is a hedge against improvements to the best known lattice attacks and it offers no protection against attacks that exploit module structure, which by design do not apply to FrodoKEM. Such an attack would still motivate an unstructured scheme. This is also what makes splitting the budget worthwhile as the hybrid of Kaiburr-1792 and FrodoKEM-976 combines a large security margin with resistance to structural attacks. Compared with FrodoKEM-1344, it has a much higher lattice attack cost and, based on its component costs, is faster.

HQC hedges against yet another possibility, a failure of the LWE assumption itself, so the comparison with HQC is one of cost only. Kaiburr-1792 is smaller than HQC-1 in combined public-key and ciphertext size and faster than HQC-5 in every operation on all three machines. For the larger parameter sets, HQC-3 and HQC-5 are faster than Kaiburr-4608 and Kaiburr-6144 in key generation and encapsulation, while decapsulation is comparable or faster for Kaiburr. Since their costs are of the same order, the choice between Kaiburr and HQC comes down to whether one believes improvements in lattice attacks are more likely than a break of the LWE assumption itself.

An important consequence of changing the module rank is that it also changes where the cost goes, especially once Keccak, which is typically the dominant cost of ML-KEM in software, is accelerated. In ML-KEM, only two steps scale quadratically with k. The first is generating the $k \times k$ matrix $\mathbf{A}$, which requires SHAKE-128 output and rejection sampling for k^2 polynomials. The second is the matrix–vector product in the NTT domain, which requires k^2 base multiplications. All other steps, including noise sampling, the NTTs, and hashing, scale linearly [Nat24b]. Our ACC profiling results demonstrate this

concretely. Matrix generation accounts for 73–84% of total cycles and the base multiplication for 8–10%, while the share of the noise sampler falls from 6.3% to 2.1%. Hardware SHAKE-128 acceleration does not remove this bottleneck either. The core waits on the accelerator for only 5.5–6.2% of total cycles, with little change across parameter sets, while the software rejection-sampling loop grows to 69.9% of total cycles for KAIBURR-6144. For an accelerated implementation, the optimization target therefore moves from symmetric throughput to rejection sampling. This is consistent with Abdulrahman et al., who also identify rejection sampling as a target for further hardware optimization [AOP+25]. In our setting, we quantify how this dominant cost grows with the module rank, at parameters far beyond those commonly studied for ML-KEM.

We make an important clarification about what our results do not intend to show. The purpose of designing and evaluating these high-security variants of ML-KEM is not to argue that such margins are currently necessary. Given the relative stability of lattice cryptanalysis and security estimates in recent years, we do not expect improvements of a magnitude that would require such parameter sets in the near future. What we characterize instead is how much margin ML-KEM can accommodate and what it costs, so that the tradeoff is understood if it ever becomes necessary.

Sia: Is this paragraph needed? This is maybe not the best place to make this argument.

Our results also have several other limitations. The security figures are estimates; for KAIBURR-4608 and KAIBURR-6144, the cheapest attacks use block sizes outside the range in which the MATZOV cost model was derived, and we obtained them with a modified estimator (Section 4). Our decryption-failure target of 2^{-128} is less strict than what ML-KEM achieves, and a stricter target would lower the largest module rank each noise parameter t supports. For the Jasmin implementation we machine-check constant-time behavior but not functional correctness. The constant-time behavior of the ACC implementation is also not machine-checked, and the ACC results are for a configuration with enlarged data memory. Finally, the hybrid of Section 8 has not been implemented, and its costs are sums of component measurements.

Several directions follow from this work. Our noise family $f(t)$ is not specific to ML-KEM, so it is an interesting question whether the same construction could extend the security margin of other module-LWE schemes, such as ML-DSA. On the implementation side, our ACC results make dedicated hardware support for rejection sampling the natural next step. It also remains open whether scaling beyond $k = 29$ is possible by modifying other parameters of ML-KEM while making minimal changes to existing implementations. A machine-checked functional-correctness and IND-CCA proof for KAIBURR extending `formosa-mlkem` [BABB+23] is also in scope. Finally, implementing the hybrid as a single KEM with a concrete combiner, and benchmarking it end to end, remains to be done.

References

[ABB+17] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Arthur Blot, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Hugo Pacheco, Benedikt Schmidt, and Pierre-Yves Strub. Jasmin: High-assurance and high-speed cryptography. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 1807–1823. ACM Press, October / November 2017. 7

[ABC+25] Gorjan Alagic, Maxime Bros, Pierre Ciadoux, David Cooper, Quynh Dang, Thinh Dang, John Kelsey, Jacob Lichtinger, Yi-Kai Liu, Carl Miller, Dustin Moody, Rene Peralta, Ray Perlner, Angela Robinson, Hamilton Silberg, Daniel Smith-Tone, and Noah Waller. Status report on the fourth round of the nist post-quantum cryptography standardization process. Technical Re-

port NIST IR 8545, National Institute of Standards and Technology, March 2025. 1

[ABD+17] Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Kyber — algorithm specifications and supporting documentation. Submission to the NIST Post-Quantum Cryptography Standardization Project, November 2017. Round 1 submission, 2017-11-30. 4, 6

[ABD+19] Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Kyber (version 2.0) — algorithm specifications and supporting documentation. Submission to round 2 of the NIST Post-Quantum Cryptography Standardization Project, March 2019. Round 2 submission, 2019-03-30. 6

[ADPS16] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key exchange - A new hope. In Thorsten Holz and Stefan Savage, editors, *USENIX Security 2016*, pages 327–343. USENIX Association, August 2016. 5

[AOP+25] Amin Abdulrahman, Felix Oberhansl, Hoang Nguyen Hien Pham, Jade Philipoom, Peter Schwabe, Tobias Stelzer, and Andreas Zankl. Towards ML-KEM & ML-DSA on OpenTitan. In Marina Blanton, William Enck, and Cristina Nita-Rotaru, editors, *2025 IEEE Symposium on Security and Privacy*, pages 1–19. IEEE Computer Society Press, May 2025. 7, 21

[APS15] Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. *J. Math. Cryptol.*, 9(3):169–203, 2015. 2, 6

[BABB+23] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Benjamin Grégoire, Vincent Laporte, Jean-Christophe Léchenet, Tiago Oliveira, Hugo Pacheco, Miguel Quaresma, Peter Schwabe, Antoine Séré, and Pierre-Yves Strub. Formally verifying Kyber: Episode IV: Implementation correctness. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023(3):164–193, June 2023. 7, 13, 15, 21

[Bar20] Elaine Barker. Recommendation for key management: Part 1 — general. NIST Special Publication 800-57 Part 1, Revision 5, National Institute of Standards and Technology, May 2020. 2

[BBF+19] Nina Bindel, Jacqueline Brendel, Marc Fischlin, Brian Goncalves, and Douglas Stebila. Hybrid key encapsulation mechanisms and authenticated key exchange. In Jintai Ding and Rainer Steinwandt, editors, *PQCrypto 2019*, pages 206–226. Springer, Cham, 2019. 2, 17

[BCD+16] Joppe W. Bos, Craig Costello, Léo Ducas, Ilya Mironov, Michael Naehrig, Valeria Nikolaenko, Ananth Raghunathan, and Douglas Stebila. Frodo: Take off the ring! practical, quantum-secure key exchange from LWE. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1006–1018. ACM, 2016. 1, 2

[BCLvV17] Daniel J. Bernstein, Chitchanok Chuengsatiansup, Tanja Lange, and Christine van Vredendaal. NTRU prime: Reducing attack surface at low cost. In Carlisle Adams and Jan Camenisch, editors, *Selected Areas in Cryptography - SAC 2017 - 24th International Conference, Ottawa, ON, Canada,*

August 16-18, 2017, Revised Selected Papers, volume 10719 of *Lecture Notes in Computer Science*, pages 235–260. Springer, 2017. 1

[BDGL16] Anja Becker, Léo Ducas, Nicolas Gama, and Thijs Laarhoven. New directions in nearest neighbor searching with applications to lattice sieving. In Robert Krauthgamer, editor, *27th SODA*, pages 10–24. ACM-SIAM, January 2016. 5

[BDK+18] Joppe W. Bos, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS - kyber: A CCA-secure module-lattice-based KEM. In *2018 IEEE European Symposium on Security and Privacy, EuroS&P 2018, London, United Kingdom, April 24-26, 2018*, pages 353–367. IEEE, 2018. 1, 2

[BKL+25] Manuel Barbosa, Matthias J. Kannwischer, Thing-Han Lim, Peter Schwabe, and Pierre-Yves Strub. Formally verified correctness bounds for lattice-based cryptography. In Chun-Ying Huang, Jyh-Cheng Chen, Shiuh-Pyng Shieh, David Lie, and Véronique Cortier, editors, *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security, CCS 2025, Taipei, Taiwan, October 13-17, 2025*, pages 156–169. ACM, 2025. 3, 14

[CMST25] Kévin Carrier, Charles Meyer-Hilfiger, Yixin Shen, and Jean-Pierre Tillich. Assessing the impact of a variant of matzov's dual attack on kyber. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2025, Proceedings, Part I*, volume 16000 of *Lecture Notes in Computer Science*, pages 444–476. Springer, 2025. 2

[CN11] Yuanmi Chen and Phong Q. Nguyen. Bkz 2.0: Better lattice security estimates. In Dong Hoon Lee and Xiaoyun Wang, editors, *Advances in Cryptology – ASIACRYPT 2011*, pages 1–20, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg. 2

[CRY] CRYSTALS-Kyber team. Security estimate and decryption-failure scripts for CRYSTALS-Kyber. https://github.com/pq-crystals/security-estimates. 5, 6

[DEEK24] Léo Ducas, Andre Esser, Simona Etinski, and Elena Kirshanova. Asymptotics and improvements of sieving for codes. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part VII*, volume 14657 of *LNCS*, pages 151–180. Springer, Cham, May 2024. 2

[DKRV18] Jan-Pieter D'Anvers, Angshuman Karmakar, Sujoy Sinha Roy, and Frederik Vercauteren. Saber: Module-LWR based key exchange, CPA-secure encryption and CCA-secure KEM. In Antoine Joux, Abderrahmane Nitaj, and Tajjeeddine Rachidi, editors, *AFRICACRYPT 18*, volume 10831 of *LNCS*, pages 282–305. Springer, Cham, May 2018. 1

[DP23] Léo Ducas and Ludo N. Pulles. Does the dual-sieve attack on learning with errors even work? In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part III*, volume 14083 of *Lecture Notes in Computer Science*, pages 37–69. Springer, 2023. 2

[DRV20] Jan-Pieter D'Anvers, Mélissa Rossi, and Fernando Virdia. (One) failure is not an option: Bootstrapping the search for failures in lattice-based encryption schemes. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part III*, volume 12107 of *LNCS*, pages 3–33. Springer, Cham, May 2020. 5

[Fro] FrodoKEM Team and Microsoft Research. PQCrypto-LWEKE: FrodoKEM. https://github.com/microsoft/PQCrypto-LWEKE. GitHub repository. 16, 17, 18, 20

[GHP18] Federico Giacon, Felix Heuer, and Bertram Poettering. KEM combiners. In Michel Abdalla and Ricardo Dahab, editors, *PKC 2018, Part I*, volume 10769 of *LNCS*, pages 190–218. Springer, Cham, March 2018. 2, 17

[GJ21] Qian Guo and Thomas Johansson. Faster dual lattice attacks for solving LWE with applications to CRYSTALS. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology - ASIACRYPT 2021 - 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6-10, 2021, Proceedings, Part IV*, volume 13093 of *Lecture Notes in Computer Science*, pages 33–62. Springer, 2021. 2

[HQC] HQC Team. HQC: Hamming Quasi-Cyclic. https://gitlab.com/pqc-hqc/hqc. GitLab repository. 16, 17, 18

[HQC25] HQC Resources Team. Hqc specifications. Documentation PDF, August 2025. Accessed: 2026-09-16. 1

[ISO26] ISO/IEC. Information technology — security techniques — encryption algorithms — part 2: Asymmetric ciphers — amendment 2. Standard ISO/IEC 18033-2:2006/Amd 2:2026, International Organization for Standardization, Geneva, Switzerland, June 2026. 1

[Laa16] Thijs Laarhoven. *Search problems in cryptography: From fingerprinting to lattice sieving*. PhD thesis, Eindhoven University of Technology, 2016. 5

[LS15] Adeline Langlois and Damien Stehlé. Worst-case to average-case reductions for module lattices. *DCC*, 75(3):565–599, 2015. 4

[MAT22] MATZOV. Report on the security of LWE: Improved dual lattice attack. Report, Zenodo, April 2022. 2, 6

[MB25] Victor Duarte Melo and Willian J. Buchanan. Kyfrog: A high-security lwe-based kem inspired by ml-kem, 2025. 2, 15

[Nat20] National Institute of Standards and Technology. Post-quantum cryptography: Round 3 submissions. https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions, 2020. Accessed: 2026-07-27. 1

[Nat24a] National Institute of Standards and Technology. Module-lattice-based digital signature standard. Technical Report Federal Information Processing Standards Publication (FIPS) 204, U.S. Department of Commerce, Washington, D.C., August 2024. 1

[Nat24b] National Institute of Standards and Technology. Module-lattice-based key-encapsulation mechanism standard. Technical Report Federal Information Processing Standards Publication (FIPS) 203, U.S. Department of Commerce, Washington, D.C., August 2024. 1, 4, 5, 20

[Nat24c] National Institute of Standards and Technology. Stateless hash-based digital signature standard. Technical Report Federal Information Processing Standards Publication (FIPS) 205, U.S. Department of Commerce, Washington, D.C., August 2024. 1

[NIS25] NIST PQC Forum. How is hqc substantially different from rlwe? NIST PQC Forum discussion, October 2025. Accessed August 29, 2026. 2

[PQ 26] PQ Code Package. mlkem-native: A unified, fast and verified mlkem library. https://github.com/pq-code-package/mlkem-native, 2026. GitHub repository, accessed September 17, 2026. 7

[SBG+23] Basavesh Ammanaghatta Shivakumar, Gilles Barthe, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Swarn Priya, Peter Schwabe, and Lucas Tabary-Maujean. Typing high-speed cryptography against spectre v1. In *2023 IEEE Symposium on Security and Privacy*, pages 1094–1111. IEEE Computer Society Press, May 2023. 13

[The26] The FrodoKEM Team. FrodoKEM: Practical quantum-secure key encapsulation from generic lattices. https://frodokem.org/#news, 2026. News section. Accessed: 2026-07-27. 1